

第21-22课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 21-22课 PDF 解析整理版

第21-22课：从复杂分析意识到Query拆解器实战

[资料] 课程总结与关系说明

核心关系：

这两节课构成了ChatBI系统从“单条SQL查询”迈向“复杂分析Agent”的关键转折点。第21课是“为什么”，建立了问题意识，明确了单SQL的能力边界和引入Agent的必要性；第22课是“怎么做”，将理论落地为代码，实现了Agent链路中的第一个核心组件——Query拆解器。

内容摘要：

- 第21课（理论基石）：
阐述了为什么“最近三个月利润为什么下降”这类问题无法用一条SQL解决。提出了复杂分析任务的三个特征（多指标拆解、交叉下钻、结论导向），并引入了Plan-and-Execute Agent的基本思路。本课的核心产出是建立判断框架：区分“查结果”（Text2SQL）与“做分析”（Agent）。
- 第22课（工程实战）：
基于第21课的认知，正式实现 query_decomposer.py。定义了“拆得好”的四个标准（可执行、有顺序、统一字段、可校验），设计了包含 question_type、analysis_goal、subtasks 的JSON输出Schema，并通过Pydantic强约束和业务依赖校验，确保拆解结果是后续Planner可信任的“中间契约”。

学习路径提示：

请先阅读第21课理解“为什么要拆”，再阅读第22课掌握“如何稳定地拆”。第22课产出的结构化子任务列表，将是第23-24课生成执行计划和管理中间结果的直接输入。

第 21 课：为什么一个 SQL 解决不了问题？复杂分析任务拆解

课程定位与目标

课程定位

本节课进入第五阶段“复杂分析能力（Agent）”的起点。前四个阶段已经解决了两个关键问题：

- 查哪里：Schema Linking 能把相关表、字段、Join 路径找出来
- 怎么算：指标 RAG 能把毛利、利润、费用率这类业务公式补进 Prompt

但这套链路默认有一个前提：用户的问题最终可以收敛成一条 SQL。——这个前提真的对吗？

这个前提在“查一个数”时成立，在“做一轮分析”时就不成立了。

[要点] 从这一课开始，系统面对的不再只是“帮我生成 SQL”，而是“帮我完成一轮复杂分析”。换言之，ChatBI 系统，绝不等同于 Text2SQL 系统。

本节课目标

- 理解单 SQL 方案的能力边界
- 识别复杂分析任务的 3 个典型特征
- 建立 Plan-and-Execute Agent 的问题意识
- 为第 22 课的 Query 拆解器实现准备统一输入

开场引入

先看两个问题：

第二个问题麻烦在哪里？

- “最近三个月”说明要看时间趋势
- “利润”说明要同时处理收入、成本、费用
- “为什么下降”说明目标不是把数据查出来，而是要做归因总结

这已经不是“写对 SQL”就结束的任务了。

[要点]

当用户的问题里同时出现“对比”“原因”“变化”“哪些拖累了”“哪个更严重”这类表达时，系统要做的就不只是查询，而是分析。

一、什么时候单 SQL 足够

单 SQL 方案最擅长的是这三类任务：

- 单指标查询：收入、订单量、客户数、费用金额
- 简单聚合分析：按区域、按产品线、按月份汇总

- 固定口径对比：同比、环比、Top N 排名

这类问题有一个共同点：输出可以直接由一次查询得到。

如果目标是“得到结果表”，这条链路已经很完整。

例如下面这个问题，本质上就是一次聚合，按客户区域汇总上个月收入总数：

```
SELECT c.region,
SUM(o.net_amount * e.rate_to_cny) AS revenue_cny
FROM sales_orders o
JOIN dim_customers c ON o.customer_id = c.customer_id
JOIN exchange_rates e ON o.order_date = e.rate_date AND o.currency = e.currency
WHERE o.order_status = 'completed'
AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY c.region;
```

系统在前四阶段已经把这类能力补得比较完整了：

- Schema Linking 负责选表、选字段、补 Join
- 指标 RAG 负责补业务公式和口径说明
- Prompt Builder 把两部分上下文组装给模型

所以在当前阶段，系统已经很适合回答“查一个数”与“查一张结果表”的问题。

1.1 单 SQL 问题的 4 个典型信号

如果一个问题同时满足下面 4 个条件，通常就还在单 SQL 的舒适区：

- 目标明确：就是要一个结果值，或者一张结果表
- 口径固定：收入、毛利率、订单量这些指标定义已经明确
- 维度有限：通常只按 1-2 个维度展开
- 没有归因要求：系统不需要回答“为什么”，只需要回答“是什么”

可以把它们理解为单 SQL 的“边界信号”：

只要问题还停留在这个层面，Text2SQL 就是高性价比方案。

1.2 引入 Agent 的额外负担

Agent 不是“更高级的默认配置”，而是“解决复杂问题的额外代价”。如果一个简单问题也强行走 Agent，多半会带来这些副作用：

- 任务拆解本身增加延迟
- 多步执行增加系统复杂度

- 中间结果管理增加出错点
- 最终回答反而可能比直接查询更绕

[要点] 不是所有问题都值得进入 Agent。能用一条 SQL 清晰回答的问题，就不应该拆成五步。

二、复杂分析任务为什么会超出单 SQL 边界

复杂分析任务通常同时具备 3 个特征。

2.1 不是查一个指标，而是拆一串指标

“利润为什么下降”表面上在问利润，实际往往要拆成：

- 收入是否下降
- 成本是否上升
- 费用是否增加
- 毛利率是否变差

单条 SQL 可以算出某个月利润，但很难直接告诉系统：

- 先看利润趋势
- 再拆收入与成本
- 然后补费用端解释
- 最后汇总成归因结论

这已经是一个多步骤分析流程。

2.2 不是看一个维度，而是要交叉下钻

“哪个市场更严重”后面通常还跟着新的问题：

- 是欧洲还是北美更严重
- 是哪个产品线拖累了整体利润
- 是哪些客户导致收入结构恶化

这意味着系统需要先得到“市场层”的结论，再继续向“产品线层”“客户层”往下钻。

单 SQL 擅长“横向取数”，不擅长“先判断，再决定下一步查什么”。

2.3 最终输出不是原始数据，而是结论

业务人员真正想要的，不是一堆结果表，而是这种表达：

[要点] 最近三个月利润下降，主要由欧洲市场储能系统产品线毛利率下滑和研发费用上升共同驱动，其中 5 月份下降最明显。

这句话至少包含四层信息：

- 时间结论：最近三个月、5 月份最明显
- 指标结论：利润下降、毛利率下滑、费用上升
- 维度结论：欧洲市场、储能系统产品线
- 归因结论：谁为主要驱动因素

这不是 SQL 天然输出的内容，而是系统对多步结果进行汇总后的产物。

2.4 下一步要查什么，取决于上一步查到了什么

复杂分析里还有一个更关键的特征：后续步骤常常依赖前面的真实结果。例如“最近三个月利润为什么下降”，系统不一定一开始就知道要查哪个区域、哪个产品线。更合理的顺序通常是：

1. 先看整体利润趋势
2. 发现 5 月份下降最明显
3. 再去看 5 月份各区域差异
4. 发现欧洲市场波动最大
5. 再继续看欧洲市场下的产品线变化

这类问题的难点，不是 SQL 不会写，而是系统需要先看一眼结果，再决定下一步往哪里钻。

这已经不是“单次生成 SQL”可以优雅处理的问题，而是一个带分支判断的分析流程。

三、为什么把 Prompt 写长一点也不够

一种常见直觉是：既然复杂问题难，那就把

Prompt

写得更详细一些。这个方向只能解决一部分问题，不能解决全部问题。

Prompt 再长，模型也仍然只是在生成一次输出。如果用户问题需要：

1. 先查趋势
2. 再查结构
3. 再查异常点
4. 最后汇总结论

那么单次生成就会遇到三个天然限制：

- 上下文冲突：一步里塞入过多目标，模型容易漏条件或混淆重点
- 中间结果不可见：模型看不到前一步真实查到了什么
- 无法动态调整：如果某一步发现异常，后续查询无法自动改变方向

这也是为什么“复杂分析任务”要从 **Text2SQL** 思维切换到 **Agent** 思维。两者最大的区别，不是模型更大，而是系统开始有了步骤意识。

四、把复杂问题硬塞成一条 SQL，会发生什么

很多系统在进入 Agent 之前，都会经历一个过渡阶段：

- 遇到复杂问题
- 不愿意改系统结构
- 于是继续把更多要求塞进 Prompt
- 希望模型“一次性想清楚并写完”

这个阶段通常能短暂提升效果，但很快就会遇到天花板。

以“最近三个月利润为什么下降”为例，如果强行要求模型一次完成全部分析，它往往会同时做这些事：

- 计算利润趋势
- 对比收入和成本
- 看费用变化
- 下钻区域和产品线
- 最后输出总结

表面上看像是一条 SQL 问题，实际上已经把 5 种目标绑在了一起。

这种硬塞方式会出现 4 类典型问题：

例如下面这类“超载 SQL”就很常见：

```
SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month, c.region, p.product_line,
SUM(o.net_amount * e.rate_to_cny) AS revenue,
SUM((p.material_cost + p.labor_cost) * o.quantity) AS cost,
SUM(o.net_amount * e.rate_to_cny) - SUM((p.material_cost + p.labor_cost) * o.quantity) AS gross_profit
FROM sales_orders o
JOIN dim_customers c ON o.customer_id = c.customer_id
JOIN dim_products p ON o.product_id = p.product_id
JOIN exchange_rates e ON o.order_date = e.rate_date AND o.currency = e.currency
WHERE o.order_status = 'completed'
AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 3 MONTH)
GROUP BY month, c.region, p.product_line;
```

这条 SQL 不是错的，但它回答不了“为什么利润下降”，只能给出一张高维结果表。后面真正难的部分还在：

- 哪个月份最异常
- 哪个区域贡献了主要下滑
- 哪个产品线拖累最大
- 成本端还是费用端是主要原因

[要点] 单条 SQL 最多是复杂分析流程里的一个步骤，而不是整个流程本身。

五、Plan-and-Execute Agent 的基本思路

从这一课开始，可以把 Agent 理解成一个“先做计划，再按计划干活”的系统。

这四步里，第 21-26 课分别解决不同环节：

- 第 21 课：先建立“为什么需要 Agent”的问题意识
- 第 22 课：先把复杂问题拆成可执行子任务
- 第 23-24 课：再把子任务转成可执行计划并管理中间结果
- 第 25-26 课：最后把多步结果汇总成分析报告

这一节只解决第一件事：为什么必须拆。

5.1 Task Decomposition

把复杂问题拆成多个可执行子任务，例如：

原始问题：最近三个月利润为什么下降？

拆分子任务：

- 先看最近三个月利润趋势
- 再看收入、成本、费用变化
- 再定位区域和产品线的异常点

5.2 Plan Generation

不是所有子任务都可以同时执行，很多任务有先后关系。

例如：

- “分析毛利变化”依赖“先拿到收入和成本”
- “识别异常区域”依赖“先算出各区域利润变化”

- “输出归因结论” 依赖 “前面的趋势与结构结果”

所以 Agent 需要知道先做什么，后做什么。

5.3 Execution

每个子任务最终仍然会回到已实现的能力上：

- Schema Linking
- 指标 RAG
- SQL 生成
- SQL 执行

这意味着 Agent 不是推翻前四阶段，而是把前四阶段当作执行器来复用。

5.4 Summary

Agent 的最后一步，不是把张表扔给用户，而是把多步结果整理成一段结论。这一步决定了系统到底是在“给数据”，还是在“给分析”。 8

六、人类如何思考“利润下降原因”

6.1 人类思考路径

在进入代码实现前，先用业务视角做一次人工拆解。

问题：[要点] 最近三个月利润为什么下降？

比较自然的拆解顺序如下：

1. 看最近三个月利润趋势
2. 对比收入、成本、费用的变化
3. 看哪些区域、产品线波动最大
4. 判断是收入端、成本端还是费用端在拖累利润
5. 汇总成一段归因结论

把它写成更像系统可执行的形式，会变成：

这时候可以明显看到：

- 子任务 3 和 4 依赖前面的趋势判断

- 子任务 5 依赖前面所有查询结果

因此任务拆解的真实目的是：把复杂问题稳定地拆成一张“任务清单 + 依赖关系图”。

6.2 通用子任务拆分方式

如果把“利润下降原因分析”继续拆得更细，往往会落成下面 4 类子任务：

这

4

类任务的意義在于：它们让系统从“直接回答问题”变成“先完成一个分析流程”。也正因为有这些任务类型，

第 22 课的 Query 拆解器才不能只返回一串文本，而必须返回带结构的信息：

- 任务名称
- 任务类型
- 依赖关系
- 维度信息
- 指标信息

七、判断框架：什么时候该进入 Agent

7.1 常见问题快速判断

如果现场面对一个新问题，可以先用下面这张表快速判断。

用这个框架看几道典型问题：

7.2 一个容易混淆的边界

“复杂 SQL”和“复杂分析”不是一回事。

- 复杂 SQL：Join 多、字段多、过滤条件多，但目标仍然明确
- 复杂分析：需要多轮判断、多步拆解、最终归因

例如：

- “各区域各产品线近三个月毛利率表”可能是复杂 SQL
- “毛利率为什么下滑，先从哪里查起”则是复杂分析

这条边界一旦分清，什么时候该进入 Agent 阶段就会非常清楚。

八、常见误区

8.1 误区一：只要模型足够强，就能一次性分析完

模型再强，也很难可靠地把“趋势、对比、下钻、归因”在一次输出里全部做好。真正需要补的不是模型智商，而是系统的执行结构。

8.2 误区二：Agent 就是循环调用 LLM

Agent 不是“多调用几次模型”，而是：

- 先明确任务边界
- 再生成执行顺序
- 再让已有 Text2SQL 能力逐步完成子任务
- 最后对结果进行汇总

没有任务结构，就没有真正的 Agent。

8.3 误区三：一进 Agent，前面的模块就没用了

恰好相反，前四阶段的模块在 Agent 阶段会变得更重要。

- Schema Linking 继续负责“查哪里”
- 指标 RAG 继续负责“怎么算”
- SQL 生成与执行继续负责“把子任务落成查询”

Agent 只是把这些能力组织起来。

小结

[要点] 单 SQL 适合“查结果”，Agent 适合“做分析”。

复杂分析任务之所以必须进入 Agent 阶段，不是因为模型不够聪明，而是因为这类问题天然需要“拆解 → 执行 → 汇总”的过程。

本节课的目标，是形成一种敏锐的判断力：

- 什么时候 Text2SQL 就够了
- 什么时候必须进入多步骤分析
- 为什么第 22 课要先做 Query 拆解器，而不是立刻去写执行器

后续课程方向：

- 实现 Query 拆解器代码

- 生成可执行计划
- 执行多条 SQL
- 管理中间结果
- 生成最终分析报告

第 22 课：从 0 实现 Query 拆解器：Task Decomposition 实战

课程定位与目标

课程定位

第 21 课已经明确了一个事实：复杂分析任务不是“一条 SQL 写得更聪明”就能解决，而是必须先拆成多个可执行步骤。本节课正式把这个判断落到代码里，目标是产出一个稳定的 `query_decomposer.py`，把复杂问题转换成结构化子任务列表（执行清单）。

本节课目标

- 定义什么样的拆解结果才算“可执行”
- 设计拆解 Prompt 与 JSON 输出 Schema
- 实现 `query_decomposer.py`
- 用真实命令运行并验证拆解效果

开场引入

先看第 21 课留下的这个问题：

[要点] 最近三个月利润为什么下降？

如果直接把它交给 SQL 生成模块，系统会遇到两个困难：

- 不知道应该先查趋势，还是先查结构
- 不知道“收入、成本、费用”这些子问题之间谁依赖谁

所以本节课的第一件事，不是生成 SQL，而是生成这样一份中间结果：

```
{
  "question_type": "profit_decline_analysis",
  "analysis_goal": "定位最近三个月利润下降的主要原因",
  "subtasks": [
    {
```

```
{
  "task_id": "task_1",
  "task_name": "明确分析时间范围与利润趋势概览",
  "depends_on": []
},
{
  "task_id": "task_2",
  "task_name": "进行利润的时间对比分析",
  "depends_on": ["task_1"]
}
]
```

只要这份结构稳定，下一课就能基于它继续生成执行计划。

一、什么叫“拆得好”

一个好的 Query 拆解结果，至少满足 4 个条件。

1.1 子任务必须可执行

不能写成“深入分析利润问题”这种空泛描述，而要写成：

- 统计最近三个月利润趋势
- 计算收入与成本变化
- 按区域拆解利润变化

也就是说，每个子任务都应该能被后续系统接住，最终落成一轮查询或一轮总结。

1.2 子任务必须有顺序

“找异常区域”通常依赖“先看趋势结果”，“输出结论”通常依赖前面所有分析结果。如果顺序不明确，Planner 就不知道哪些步骤可以先执行，哪些必须等前一步完成。

1.3 子任务必须有统一字段

为了让后续模块直接复用，输出格式必须固定。本节统一为这 7 个字段：

- task_id
- task_name
- task_type
- description
- depends_on
- dimensions

- metrics

1.4 子任务必须可校验

如果模型返回了下面这种结果，系统就无法往下走：

- task_3 依赖 task_9
- depends_on 里写了不存在的任务
- 顶层字段缺失

二、拆解器定位

拆解器在整条 Agent 链路里的位置：

拆解器的职责：把复杂问题拆成一组有顺序、有依赖、可执行的子任务。

拆解器和前四阶段的 Text2SQL 模块关系：

- 第 22 课新增的是任务拆解层
- 第 6-20 课构建的是执行能力层

Agent 不是重写 Text2SQL，而是在它上面多了一层“先想后做”的编排能力。

三、输出 Schema 设计

3.1 顶层结构

本节统一输出 3 个顶层字段：

```
{
  "question_type": "profit_decline_analysis",
  "analysis_goal": "定位最近三个月利润下降的主要原因",
  "subtasks": []
}
```

字段作用如下：

3.2 单个子任务结构

```
{
  "task_id": "task_3",
  "task_name": "拆解利润变化的指标驱动因素",
  "task_type": "metric_decomposition",
  "description": "从利润=收入-成本的角度拆解利润下降来源",
  "depends_on": ["task_1", "task_2"],
  "dimensions": ["月份", "指标项"],
```

```
"metrics": ["利润", "收入", "成本"]
}
```

dimensions 和 metrics 的目的：

- metrics 告诉系统这一子任务要关注哪些指标
- dimensions 告诉系统这一子任务要按什么维度展开

这两个字段会直接影响下一步的计划生成。

四、如何让模型按约束拆任务

本节设计的 Prompt 专门用于问题拆解，核心思路只做三件事：

1. 明确角色：你是 Query 拆解器，不是 SQL 生成器
2. 明确格式：必须返回 JSON
3. 明确约束：依赖只能引用前面已经出现的任务

核心 Prompt 如下：

```
def build_decomposition_prompt(user_question: str) -> tuple[str, str]:
    system_msg = (
        "你是企业级 ChatBI 系统中的任务拆解器。"
        "请把复杂分析问题拆成可执行的子任务列表，"
        "输出必须是 JSON，不要输出额外解释。"
    )
    prompt = f"""
```

请将下面的复杂分析问题拆解为结构化子任务，并严格输出 JSON：

用户问题：{user_question}

输出要求：

1. 顶层字段包含 question_type、analysis_goal、subtasks
 2. subtasks 是有序数组，每个任务必须包含：
 - task_id
 - task_name
 - task_type
 - description
 - depends_on
 - dimensions
 - metrics
 3. task_id 使用 task_1、task_2 这类格式
 4. depends_on 只能引用前面已经出现的 task_id
 5. 如果问题涉及时间对比、维度对比、指标拆解，请显式拆成多个子任务
 6. 仅返回 JSON 对象，不要使用 Markdown
- ```
""".strip()
 return system_msg, prompt
```

这段 Prompt 的重点不是“让模型更聪明”，而是把自由度收窄到后续模块真正需要的范围里。

[要点]

如果没有字段约束，模型返回的就不是“系统契约”，而是一段看起来很像答案、却无法继续编排的自然语言说明。

## 五、代码实现：query\_decomposer.py

### 5.1 Pydantic 输出强约束

[要点] 注意：虽然现在最新的成型框架（如 langchain 等）已经可以直接强制模型输出为固定的 python class 数据结构，但底层仍然是和 Pydantic 解析类似的约束解析，这里我们直接使用原始方式，更有助于理解本质。

经典面试题：如何让 LLM 输出固定结构的内容？

先定义两层模型：

```
class DecomposedTask(BaseModel):
 task_id: str
 task_name: str
 task_type: str
 description: str
 depends_on: list[str] = Field(default_factory=list)
 dimensions: list[str] = Field(default_factory=list)
 metrics: list[str] = Field(default_factory=list)

class DecompositionPlan(BaseModel):
 question_type: str
 analysis_goal: str
 subtasks: list[DecomposedTask]
```

这样做的价值很直接：

- JSON 字段缺了，立刻报错
- 字段类型不对，立刻报错
- 后续拿到的是稳定结构，不是松散字典

### 5.2 主流程

对应到代码里，就是下面这段主链路：

```
class QueryDecomposer:
 def decompose(self, user_question: str) -> dict:
 question = user_question.strip()
 if not question:
 raise ValueError("输入问题不能为空")

 system_msg, prompt = build_decomposition_prompt(question)
 raw_response = self.response_generator(system_msg, prompt)
 plan = self._parse_plan(raw_response)
 self._validate_dependencies(plan)
```

```
return plan.model_dump()
```

### 5.3 为什么要单独做依赖校验

Pydantic 能保证 JSON 结构正确，但它不知道：

- task\_3 依赖的 task\_9 是否存在
- 依赖是否引用了后面的任务

所以还要补一层业务校验：

```
@staticmethod
def _validate_dependencies(plan: DecompositionPlan) -> None:
 seen_ids: set[str] = set()
 for task in plan.subtasks:
 for dep_id in task.depends_on:
 if dep_id not in seen_ids:
 raise ValueError(f"任务 {task.task_id} 依赖了不存在的任务: {dep_id}")
 seen_ids.add(task.task_id)
```

这段逻辑很轻，但意义很大：

- 它把“看起来像能用”的结果，变成“后续模块可以直接信任”的结果
- 它让第 23 课的 Planner 可以直接基于 depends\_on 生成执行顺序

## 七、运行验证

### 7.1 环境信息

本次实测环境：

- 工作目录：code/chatbi\_mvp
- uv 版本：0.8.12
- Python 版本：3.12.9
- 当前模型：gpt-5.4

### 7.2 运行命令

```
uv run python query_decomposer.py "最近三个月利润为什么下降?"
```

输出节选：

```
{
 "question_type": "profit_decline_analysis",
 "analysis_goal":
 "定位最近三个月利润下降的主要原因，明确是收入下降、成本上升、结构变化还是异常波动导致，并识别关键时间段、业务维度和核心影响因素。",
```

```

"subtasks": [
 {
 "task_id": "task_1",
 "task_name": "明确分析时间范围与利润趋势概览",
 "task_type": "time_trend_analysis",
 "depends_on": [],
 "dimensions": ["月份"],
 "metrics": ["利润", "收入", "成本", "利润率"]
 },
 {
 "task_id": "task_2",
 "task_name": "进行利润的时间对比分析",
 "task_type": "time_comparison",
 "depends_on": ["task_1"],
 "dimensions": ["月份", "对比类型（环比/同比）"],
 "metrics": ["利润", "利润变动额", "利润变动率"]
 }
]
}

```

再看一个更复杂的问题：

```
uv run python query_decomposer.py "对比欧洲市场和北美市场最近三个月的毛利率变化，并找出下滑更严重的产品线"
```

输出节选：

```

{
 "question_type": "comparative_trend_analysis",
 "analysis_goal": "对比欧洲市场和北美市场最近三个月的毛利率变化，识别两个市场中毛利率下滑更严重的产品线，并明确下滑幅度与变化趋势。",
 "subtasks": [
 {
 "task_id": "task_2",
 "task_name": "提取欧洲市场最近三个月毛利率数据",
 "depends_on": ["task_1"]
 },
 {
 "task_id": "task_3",
 "task_name": "提取北美市场最近三个月毛利率数据",
 "depends_on": ["task_1"]
 },
 {
 "task_id": "task_9",
 "task_name": "识别下滑更严重的产品线",
 "depends_on": ["task_8"]
 }
]
}

```

这说明拆解器已经能稳定识别：

- 时间范围定义
- 市场维度对比
- 产品线层下钻

- 依赖链顺序

## 小结

[要点] 本节课交付的，是一份稳定的中间契约：

复杂问题 -> 结构化子任务列表 -> 依赖关系

有了这份契约，下一课 Planner 才能继续回答两个问题：

- 哪些任务可以先执行
- 哪些任务必须等待前面的结果

如果没有这一步，Agent 只能“想到哪查到哪”；有了这一步，系统才开始真正具备计划能力。

## 附录：完整代码路径

- Query 拆解器模块：query\_decomposer.py

附件（飞书无下载权限）：第 21 课：为什么一个 SQL 解决不了问题？复杂分析任务拆解(1).pdf

附件（飞书无下载权限）：第 22 课：从 0 实现 Query 拆解器：Task Decomposition 实战(1).pdf